The **full-stack** deployment course

From bare-metal cloud infrastructure to automated production-ready pipelines

2026

Imad Saddik

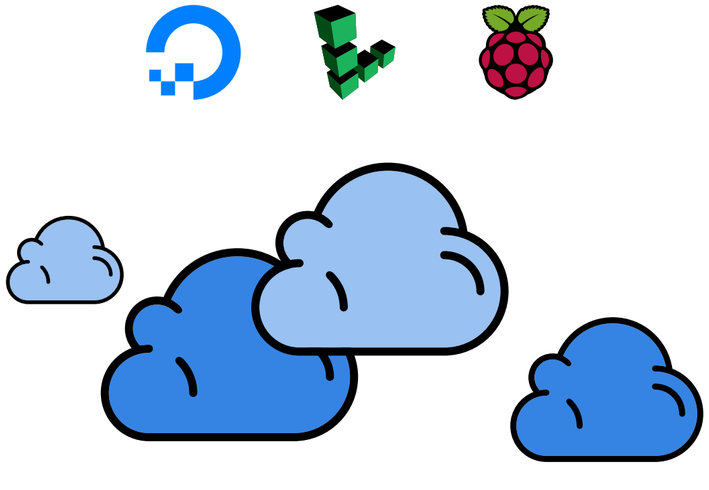
The full-stack deployment course

Problem

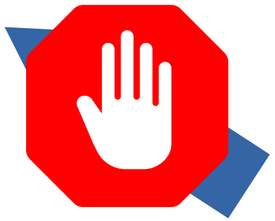


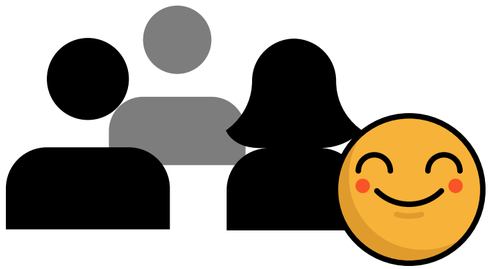
Web application

Source code



Production

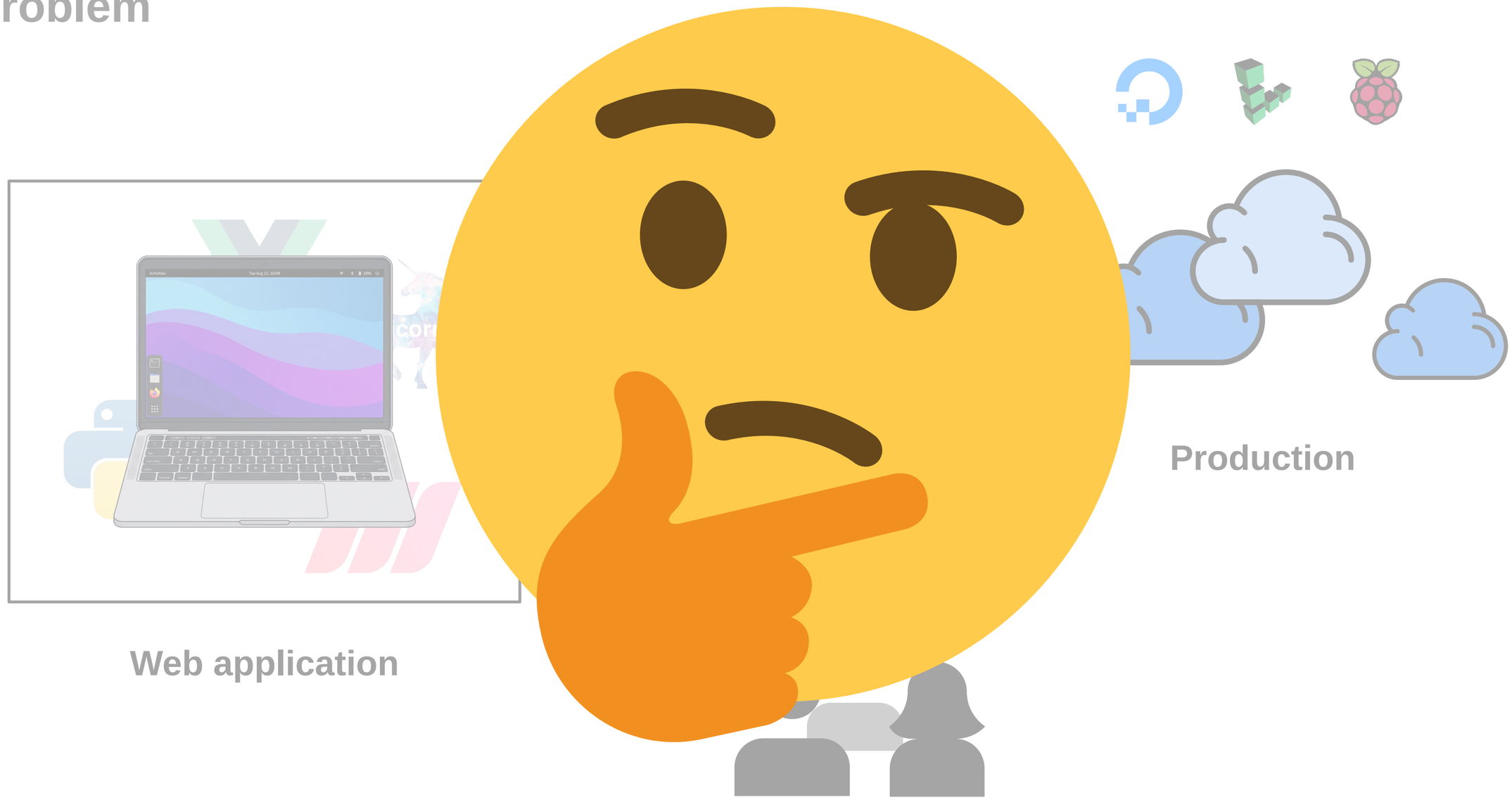




Users

The full-stack deployment course

Problem



Web application

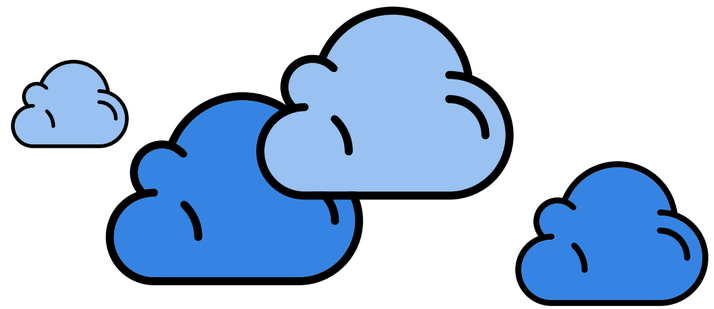
Production

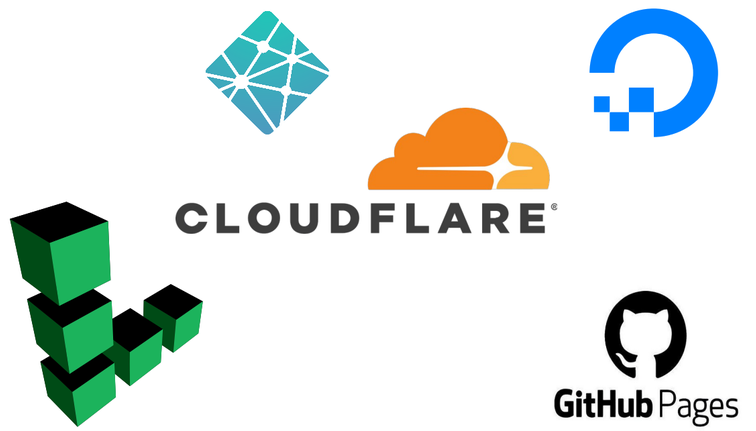
Users

The full-stack deployment course

Solution

- Host application source code on **cloud-rented** virtual or physical instances.
- Managed platforms **simplify** resource allocation and deployment pipelines.
- High abstraction **masks** underlying mechanics, complicating root-cause troubleshooting when failures occur.
- Best suited for developers with a **practical understanding** of Linux, networking, and bare-metal environments.





The full-stack deployment course

Behind the scenes

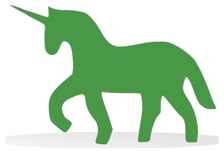




Operating system







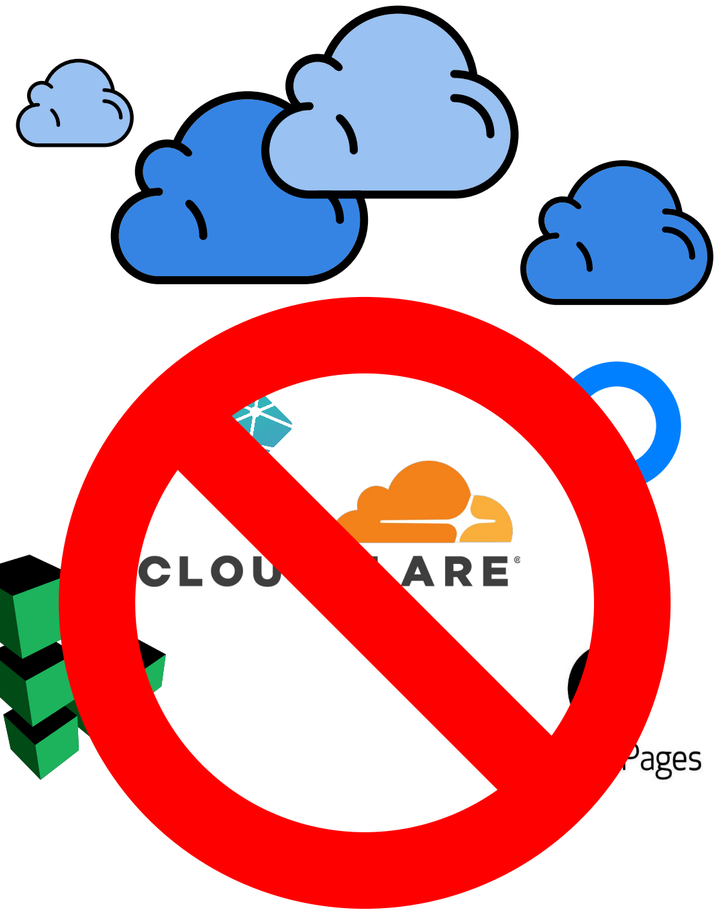
Software





Security

...



The full-stack deployment course

Core philosophy

> *“Before you automate anything, you must understand how to execute it manually”*
>
> Imad Saddik

The full-stack deployment course

Real-world foundation

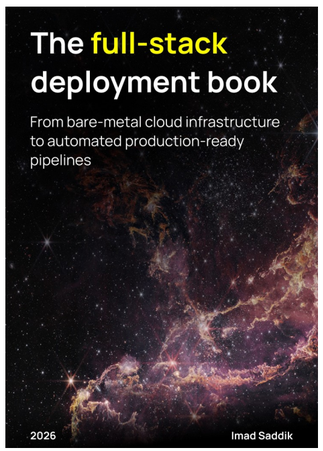


The full-stack deployment course

Real-world foundation



- Distilled from 8 months of active refinement; every script, concept, and config was tested, broken, and optimized on a live server.
- Built around a production app using **Vue.js** (Frontend), **FastAPI** (Backend), and self-hosted **Meilisearch**.



The full-stack deployment course

An overview of the 6 course modules

- **Module 1:** The foundation
  - Cloud infrastructure provisioning
  - SSH key authentication and shortcuts
  - Locked rootless user model
  - Disabling root login and password auth
  - Firewall configuration and Fail2Ban protection



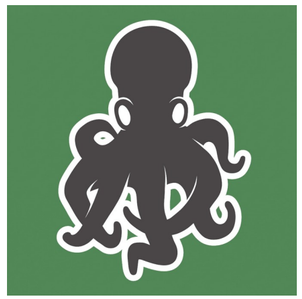
The full-stack deployment course

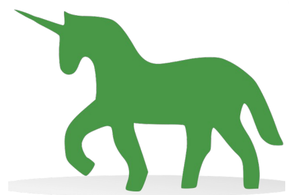
An overview of the 6 course modules

- **Module 2:** The application runtime
  - Code upload and production directory layout
  - Python backend and virtual environment setup
  - Swap memory configuration for low-RAM builds
  - Backend process management with Gunicorn and Supervisor
  - Nginx reverse proxy routing via Unix sockets
  - HTTP security headers and Content Security Policy (CSP)

OOM









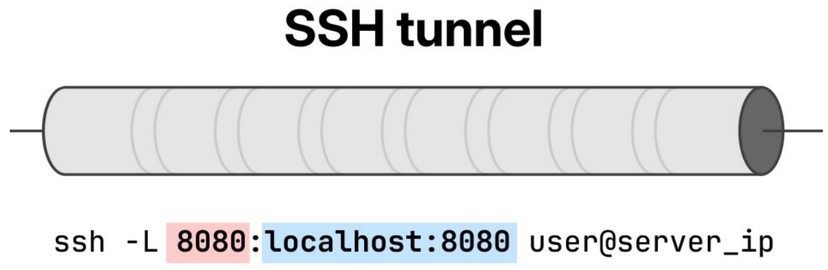
The full-stack deployment course

An overview of the 6 course modules

- **Module 3:** Data & search
  - Data export and dump migration strategy
  - Meilisearch binary installation and setup
  - System user creation with restricted shell access
  - Background service management via systemd
  - Automated daily snapshot backups configuration
  - Secure admin management via SSH tunnels and GUI







The full-stack deployment course

An overview of the 6 course modules

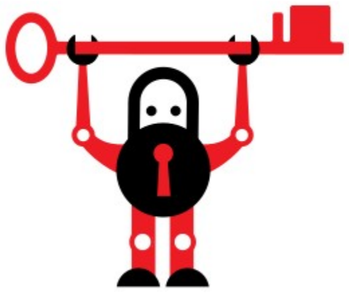
- **Module 4:** Global delivery & app security
  - Domain registration and DNS record management
  - Automated Let's Encrypt SSL/TLS certificate deployment with Certbot
  - Automated certificate renewal configuration via cron and post-renewal hooks
  - Global CDN integration and SSL/TLS encryption setup with Cloudflare
  - Visitor real IP restoration at the Nginx server level
  - ...







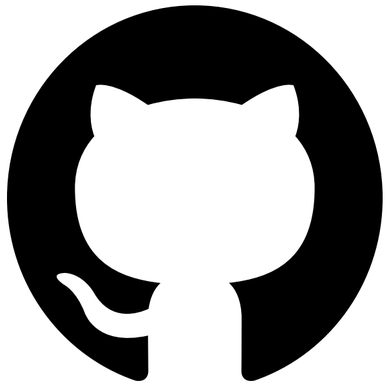




The full-stack deployment course

An overview of the 6 course modules

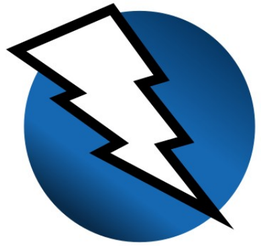
- **Module 5:** The automation pipeline
  - Branch protection rules setup and default branch hardening
  - Local code formatting and linting via Git pre-commit hooks
  - Modular CI pipeline architecture using reusable GitHub Actions workflows
  - Automated unit testing, dependency auditing (SCA), and code scanning (SAST)
  - Live integration testing, Playwright E2E testing, and OWASP ZAP DAST scans
  - ...





Pre-Commit





The full-stack deployment course

An overview of the 6 course modules

- **Module 6:** Optimization & maintenance
  - Traffic analytics with GoAccess and Nginx logs
  - Real-time system resource monitoring and process tracking using Btop
  - Traffic anomaly detection and outlier IP banning via UFW firewall rules
  - Backend API load testing and performance benchmarking using Locust
  - Disk usage analysis with NCDU and systemd journal storage restriction
  - Log retention policy updates and automated monthly system cleanup scripts









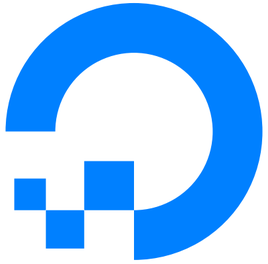
Module 1: The foundation

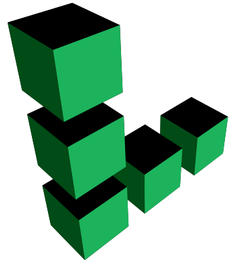
Cloud setup & access

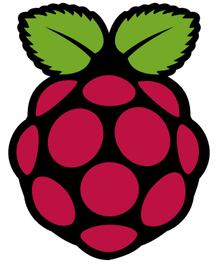
Cloud setup & access

Prerequisites

- **Goal:** A secure environment before deploying code.
- **Providers:** OVHcloud VPS, AWS EC2, bare metal, or Raspberry Pi.







Cloud setup & access

Prerequisites

• **Goal:** A secure environment before deploying code.

• **Providers:** OVHcloud VPS, AWS EC2, bare metal, or Raspberry Pi.

Learning objectives

• Deploy a server running **Ubuntu 24.04 LTS**.

• Generate and configure **SSH Keys** (Ed25519).

• Understand the **"Lock & Key"** security model.

• **Security:** Disable password authentication completely.

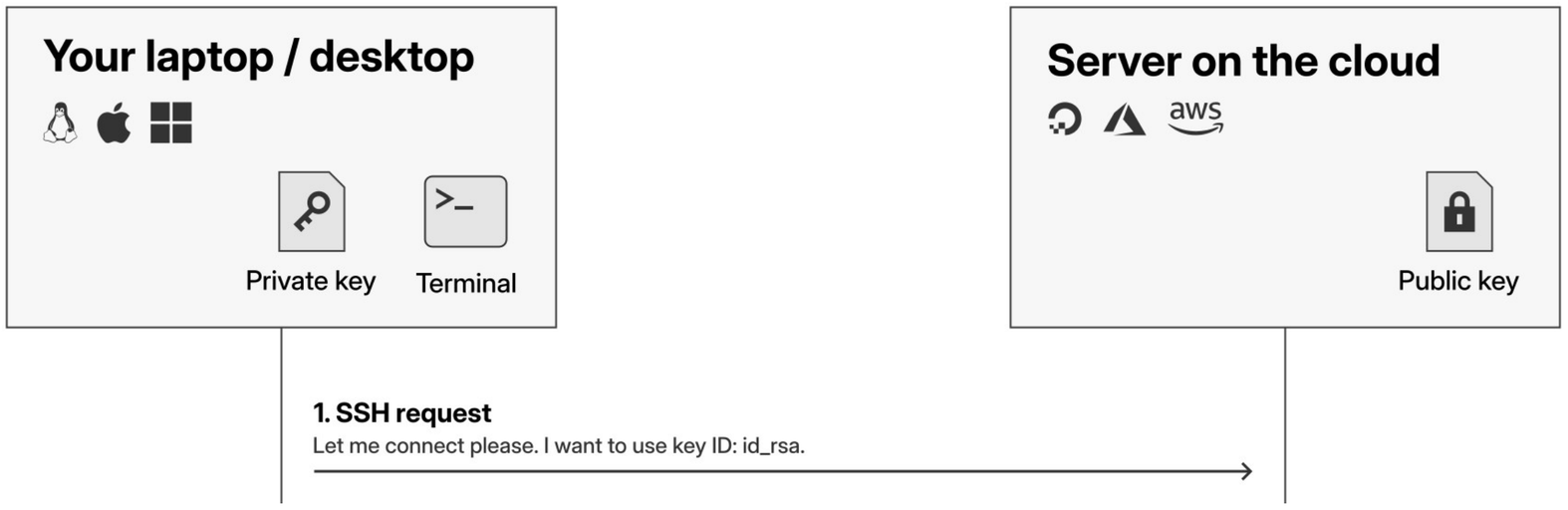


How SSH keys work

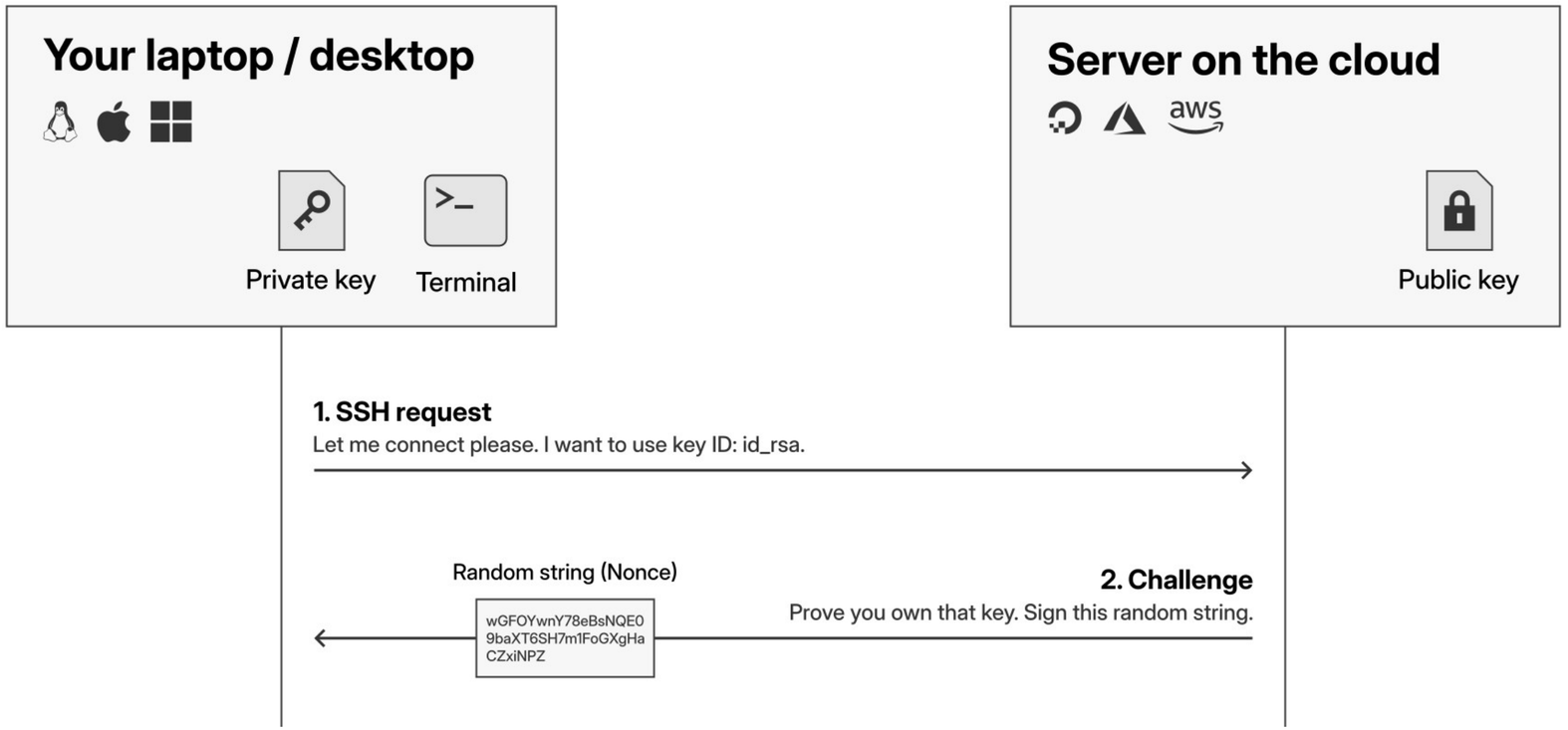




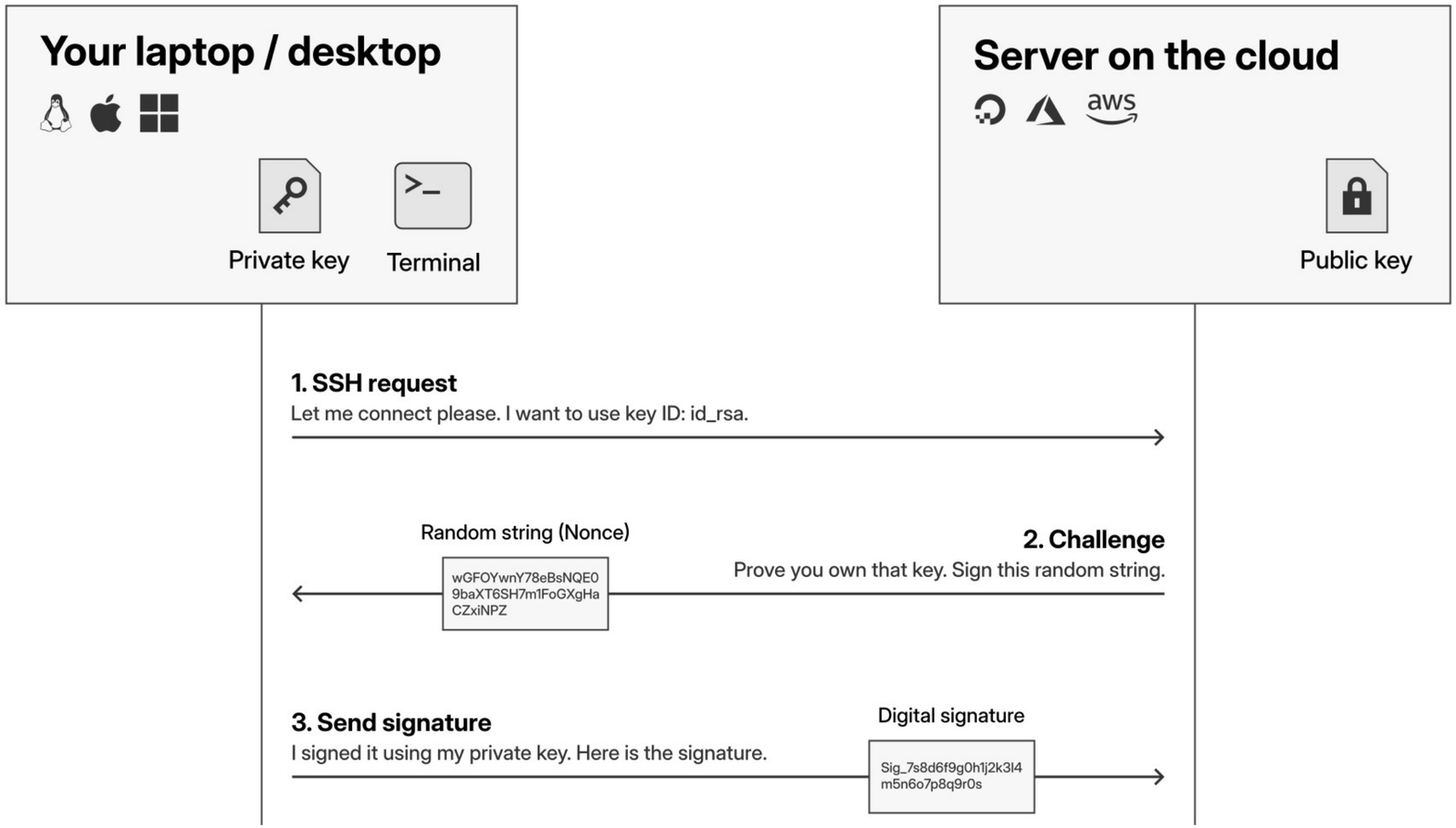
How SSH keys work



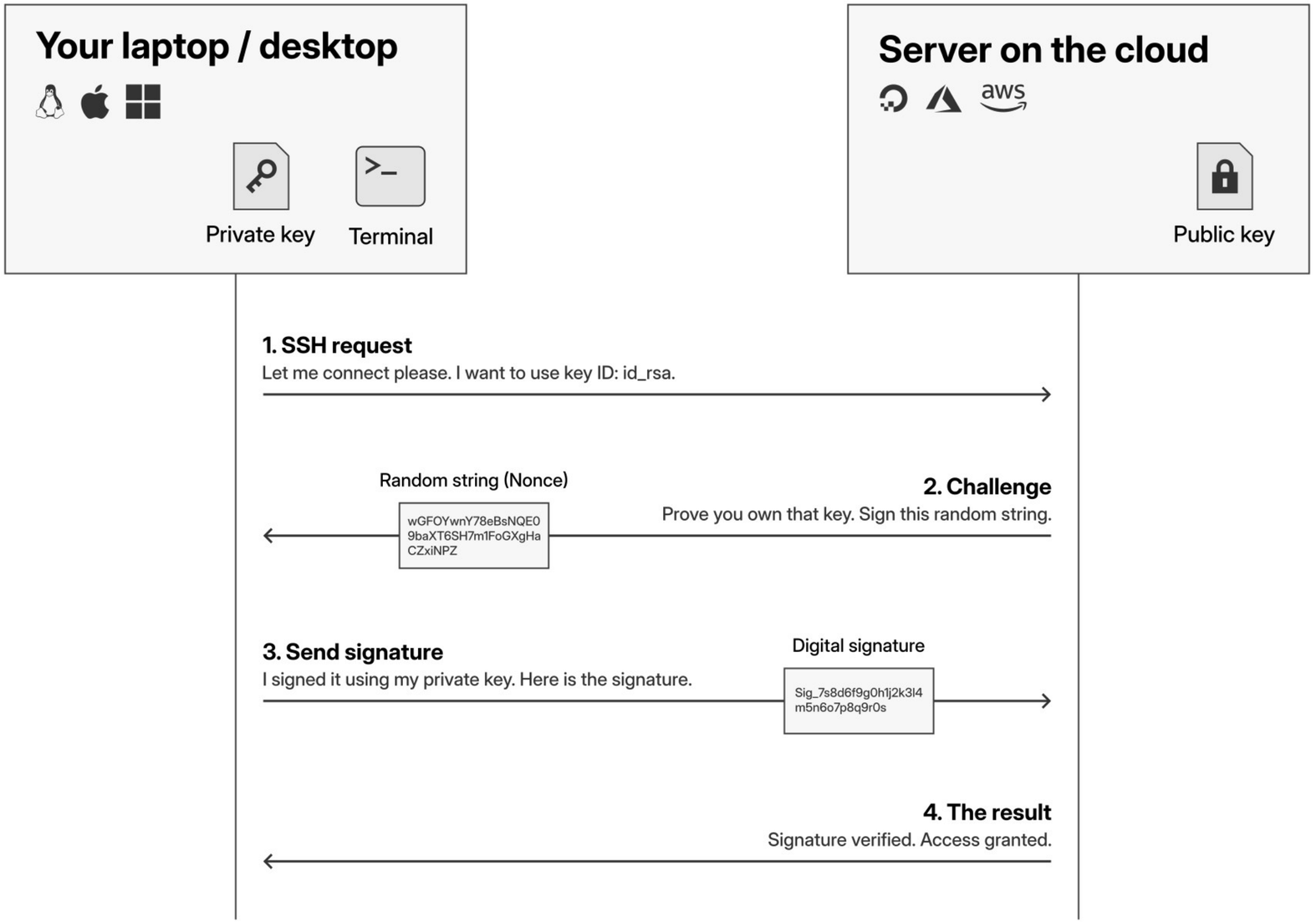
How SSH keys work



How SSH keys work



How SSH keys work



What is next?

- **Current status:** You now have a clean, secure Ubuntu server.
- **The risk:** The server is still exposed to the open internet.
- **Next steps:**
  - **Firewall configuration:** Setting up UFW (Uncomplicated Firewall).
  - **Brute force protection:** Installing Fail2Ban to block attackers.
  - **Emergency access:** How to use the recovery console.



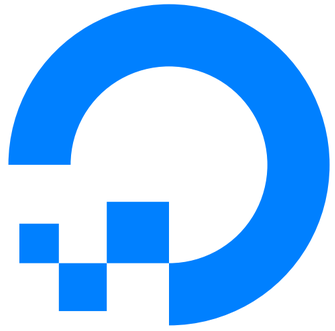


Module 1: The foundation

The firewall strategy

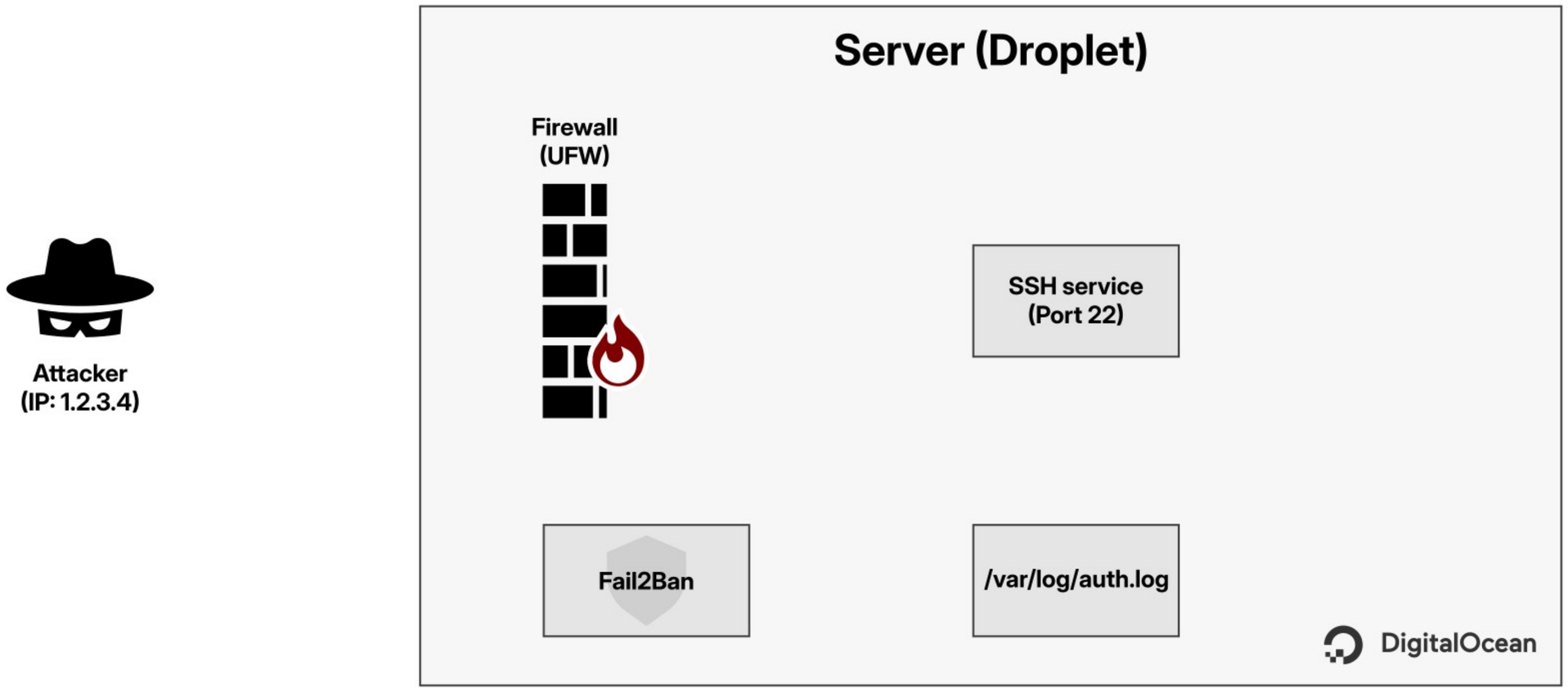
The firewall strategy

- **Problem:** Default Linux servers accept traffic on all ports.
- **Solution:** Use UFW to block everything by default.
- **Automation:** Use Fail2Ban to block bots instantly.
- **Safety net:** Using the OVHcloud KVM console and rescue mode.

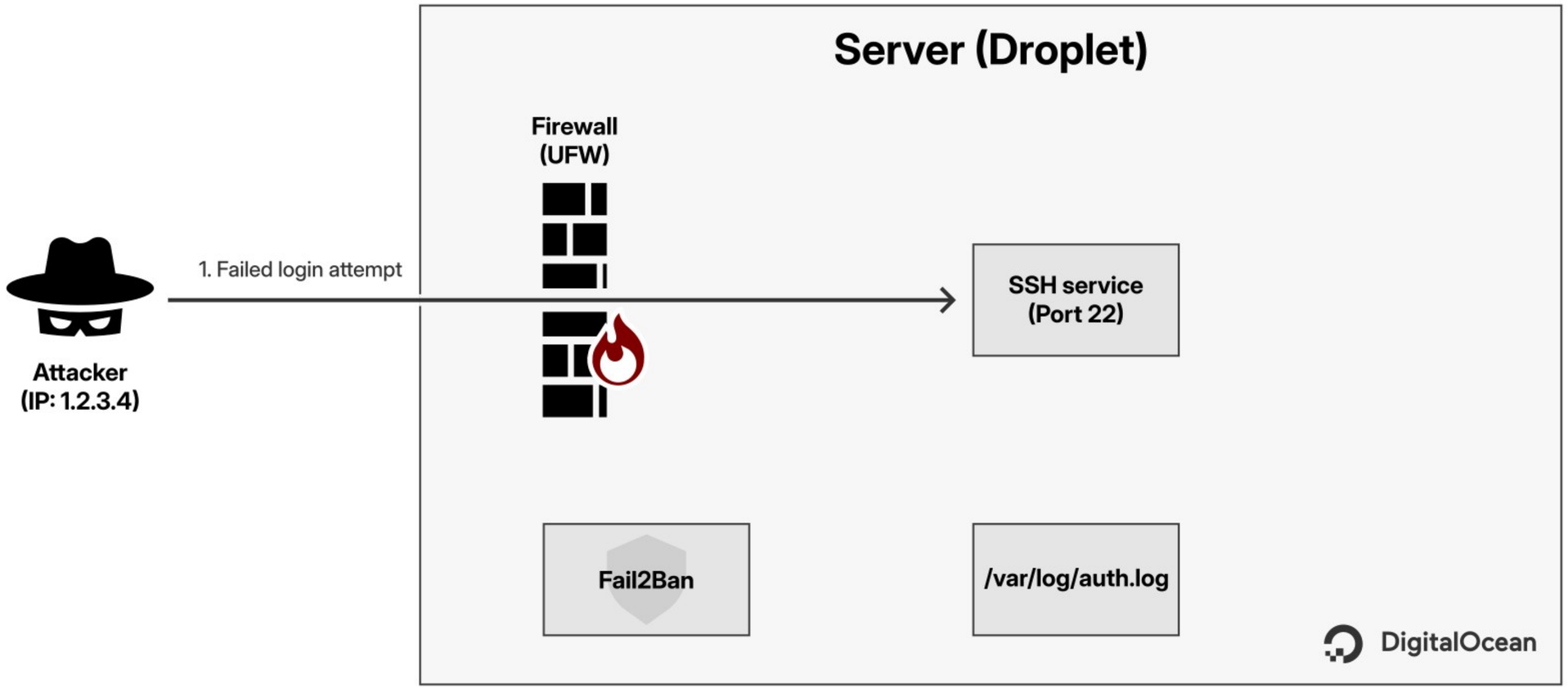




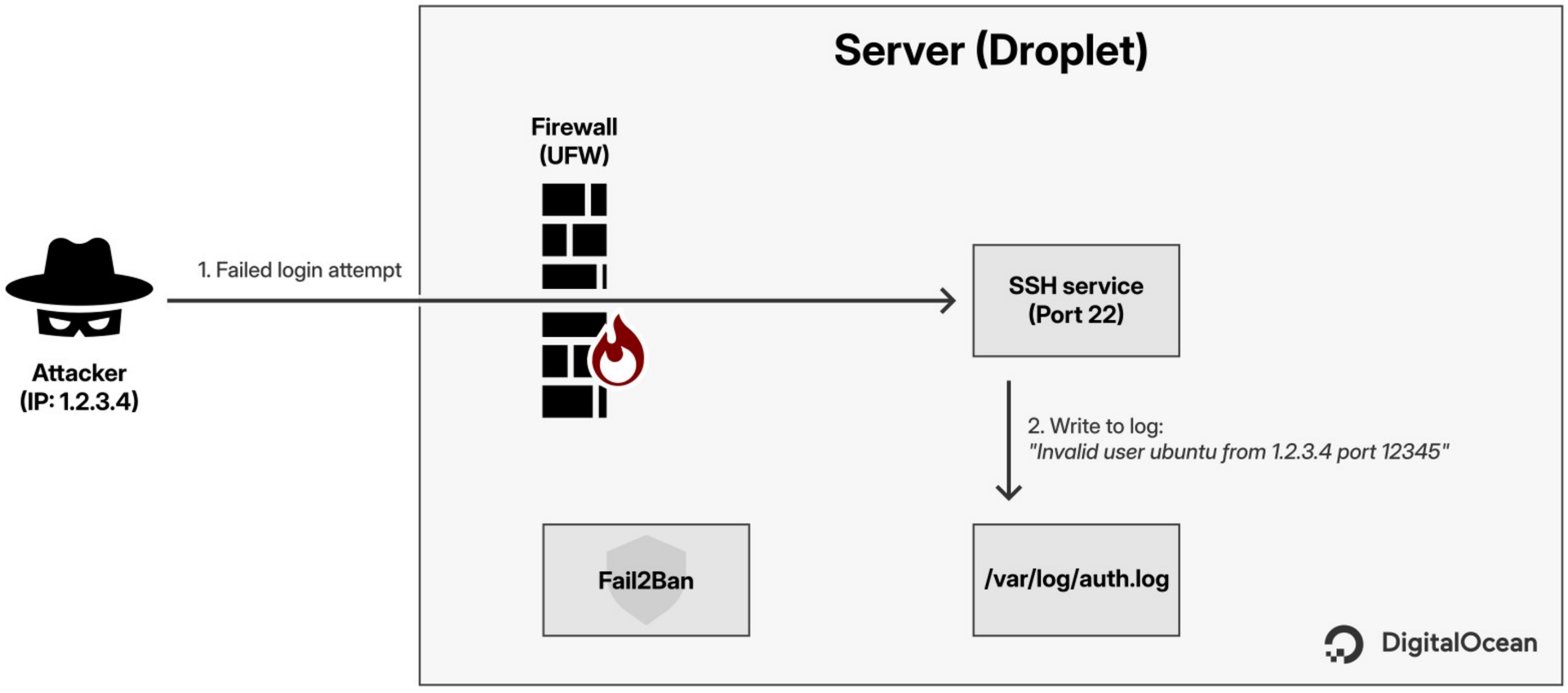
How Fail2Ban works



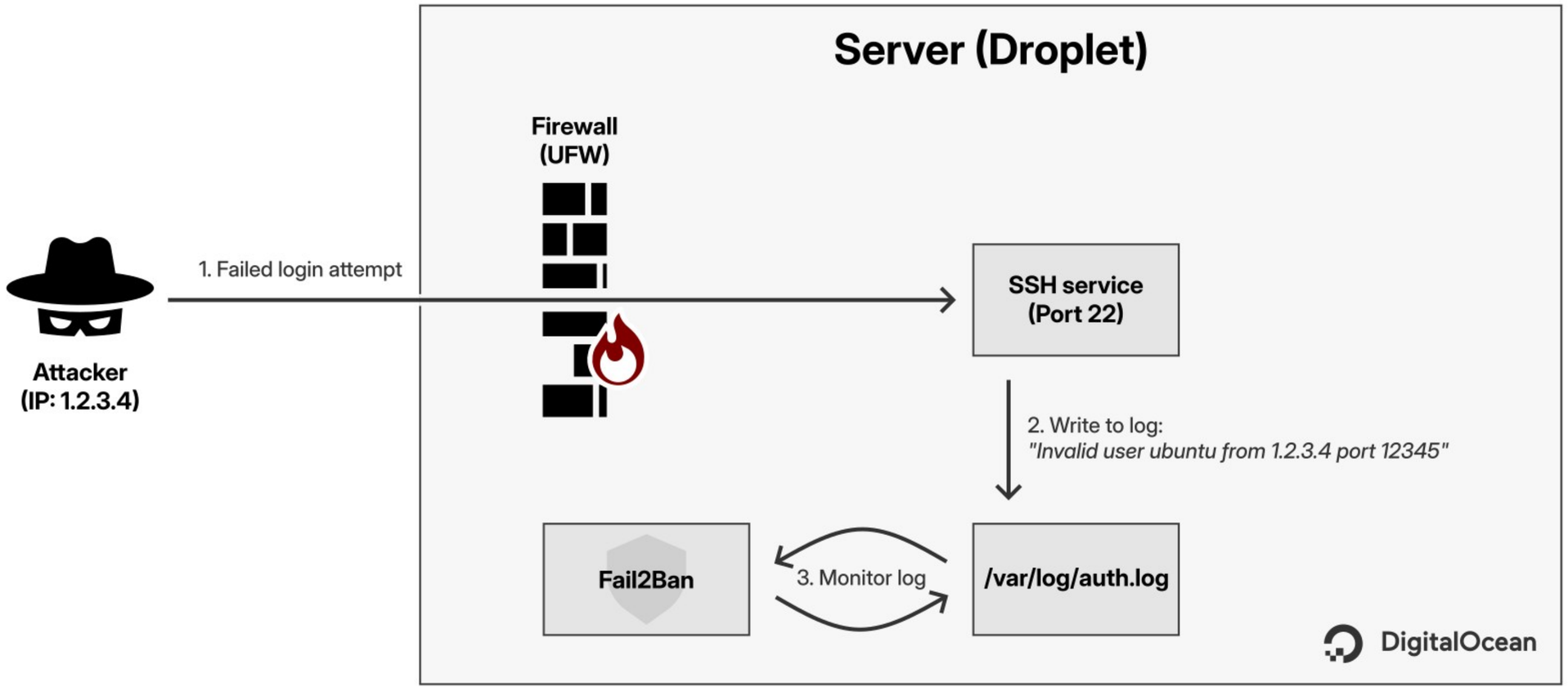
How Fail2Ban works



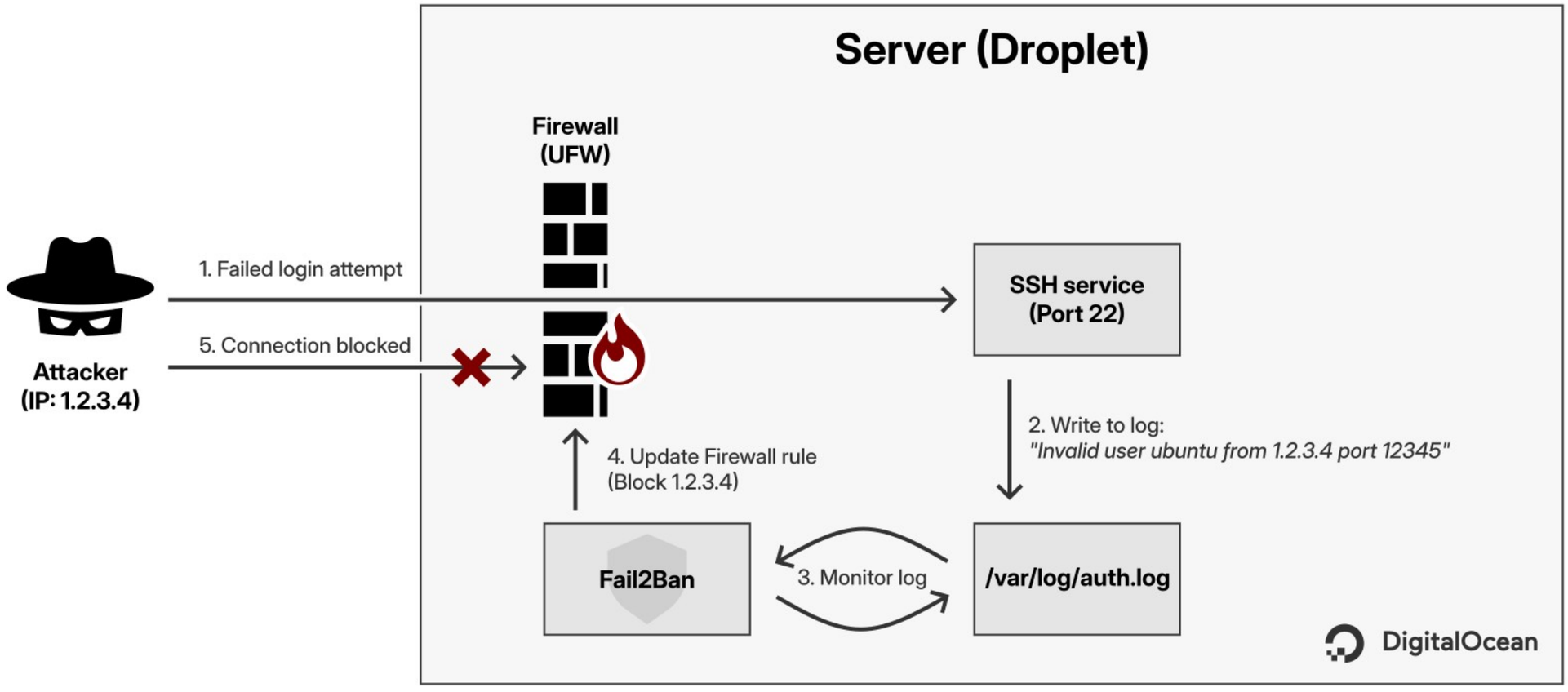
How Fail2Ban works



How Fail2Ban works



How Fail2Ban works

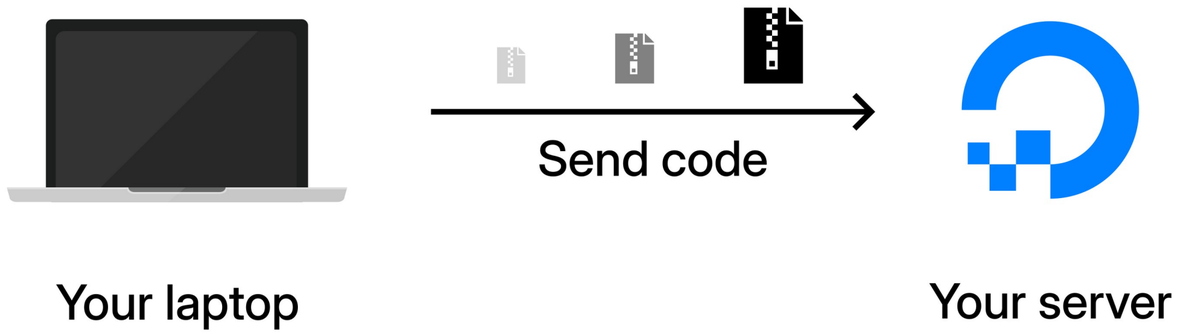


Module 2: The application runtime

Manual deployment

Manual deployment

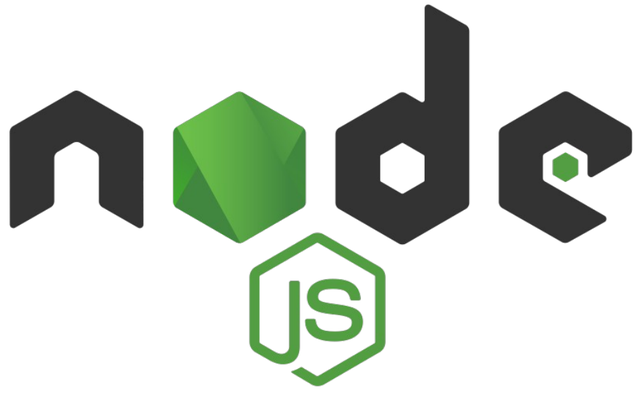
- Move code using **scp** and organize permissions.



Manual deployment

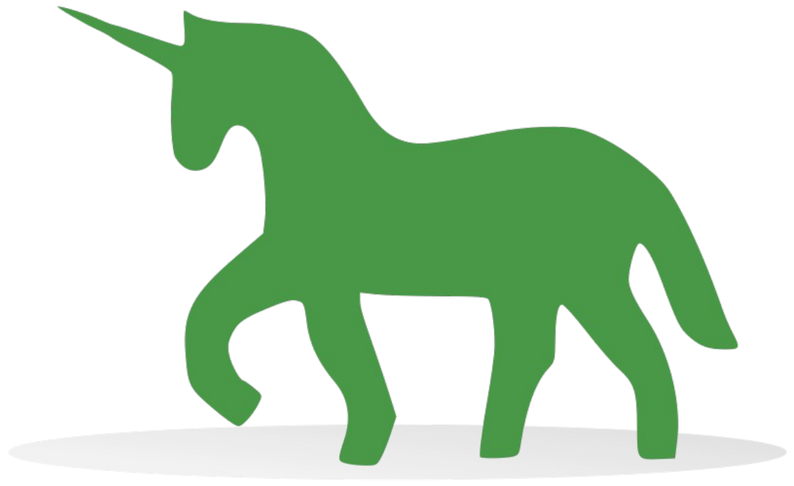
- Move code using **scp** and organize permissions.
- Install **Python** (Backend) and **Node.js** (Frontend).
- Fix build errors using **Swap files**.
- Use **SSH Tunnels** to bypass the **firewall**.



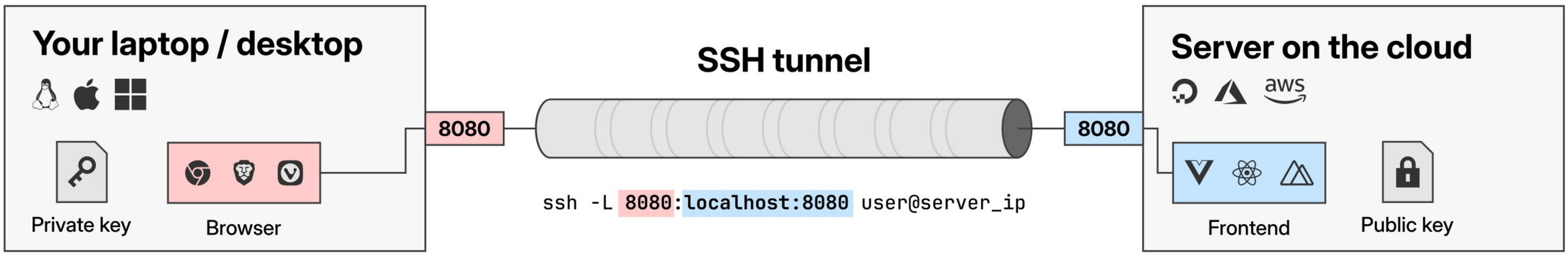


Manual deployment

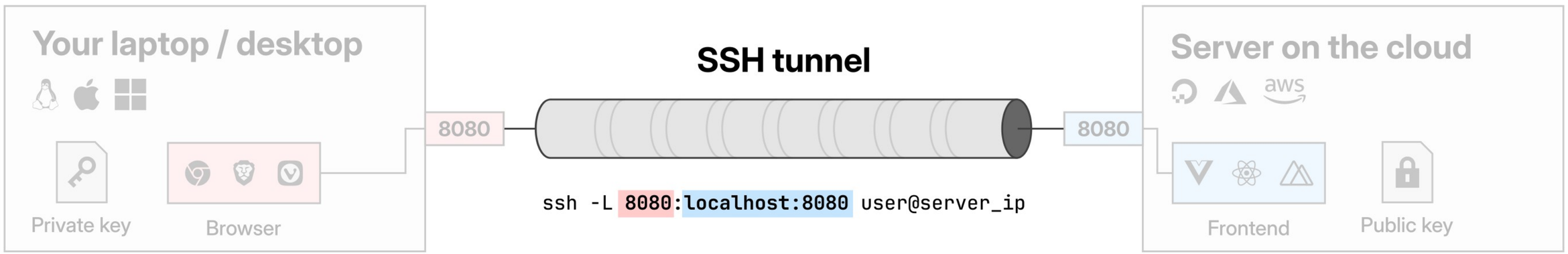
- Move code using **scp** and organize permissions.
- Install **Python** (Backend) and **Node.js** (Frontend).
- Fix build errors using **Swap files**.
- Use **SSH Tunnels** to bypass the **firewall**.
- Configure **Gunicorn** and **Supervisor**.



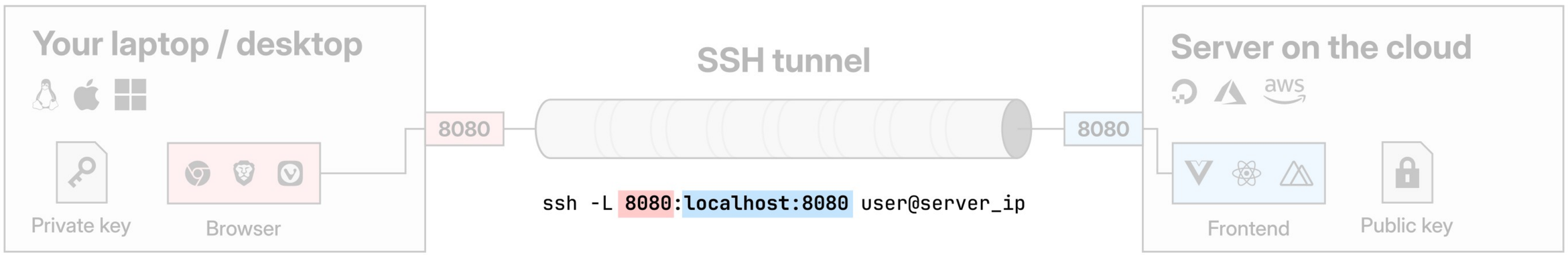
SSH tunneling



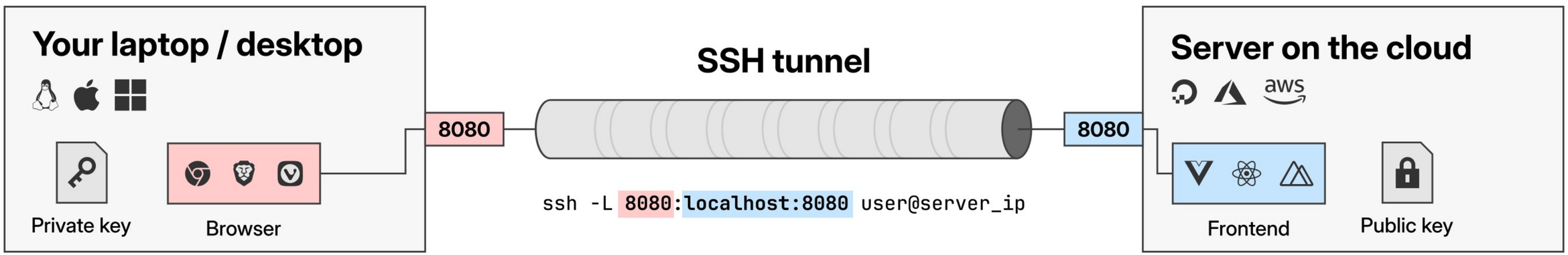
SSH tunneling



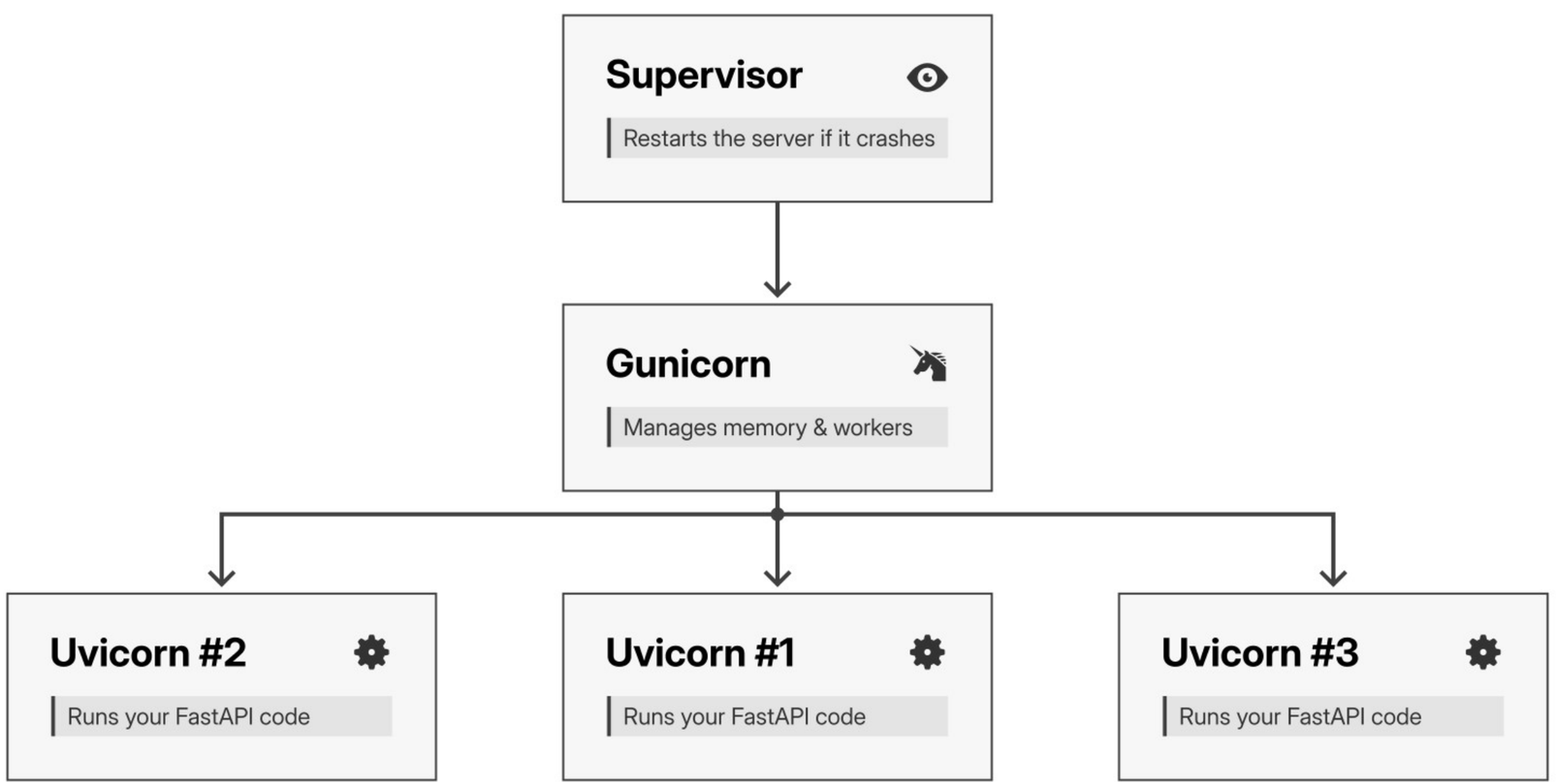
SSH tunneling



SSH tunneling



Keeping the app alive



Module 2: The application runtime

Reverse proxy & security headers

Reverse proxy & Security

- Sync frontend and backend URLs.
- Install and configure **Nginx**.



Reverse proxy & Security

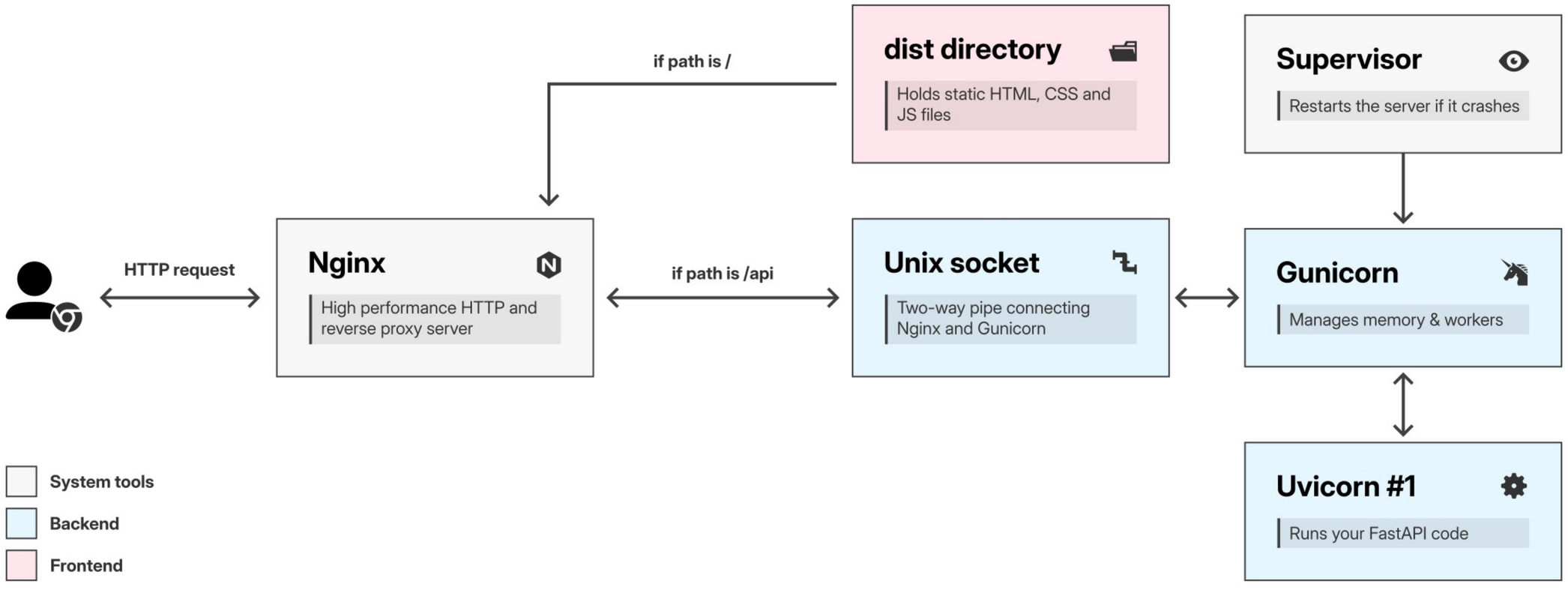
- Sync frontend and backend URLs.
- Install and configure **Nginx**.
- Understand and fix the **502 Bad Gateway** error.



Reverse proxy & Security

- Sync frontend and backend URLs.
- Install and configure **Nginx**.
- Understand and fix the **502 Bad Gateway** error.
- Protect users with **HTTP Security Headers (CSP)**.

Nginx: The traffic cop



Module 3: Data and search

Self-hosting Meilisearch

Self-hosting a search engine

- **The engine**: Meilisearch (a lightning-fast Rust binary).
- **Security**: Isolate the database using a **system user**.
- **Automation**: Run it as a background service using **systemd**.
- **Maintenance**: Schedule automatic snapshot backups.
- **Management**: Use an **SSH Tunnel** to access a visual dashboard securely.



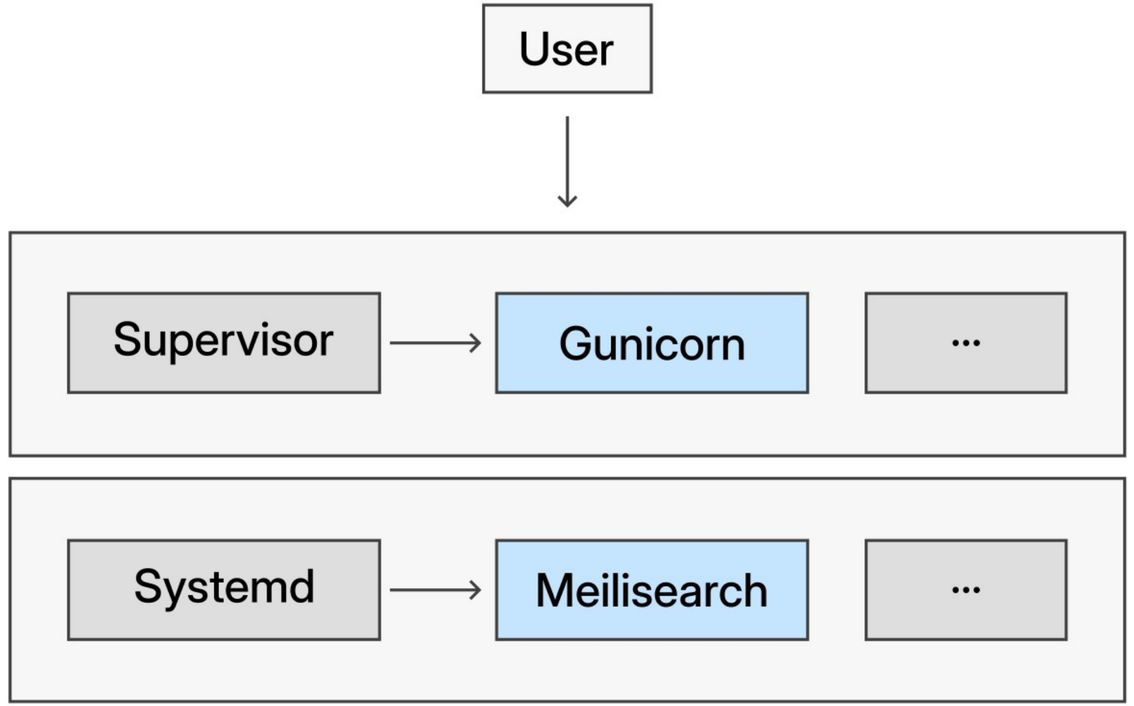
What is a system user?

Feature	Standard user	System user
Purpose	Human administration	Running background services
Shell	/bin/bash	/bin/false
Login	Yes	No
Home folder	/home/user	/var/lib/meilisearch

Systemd vs Supervisor

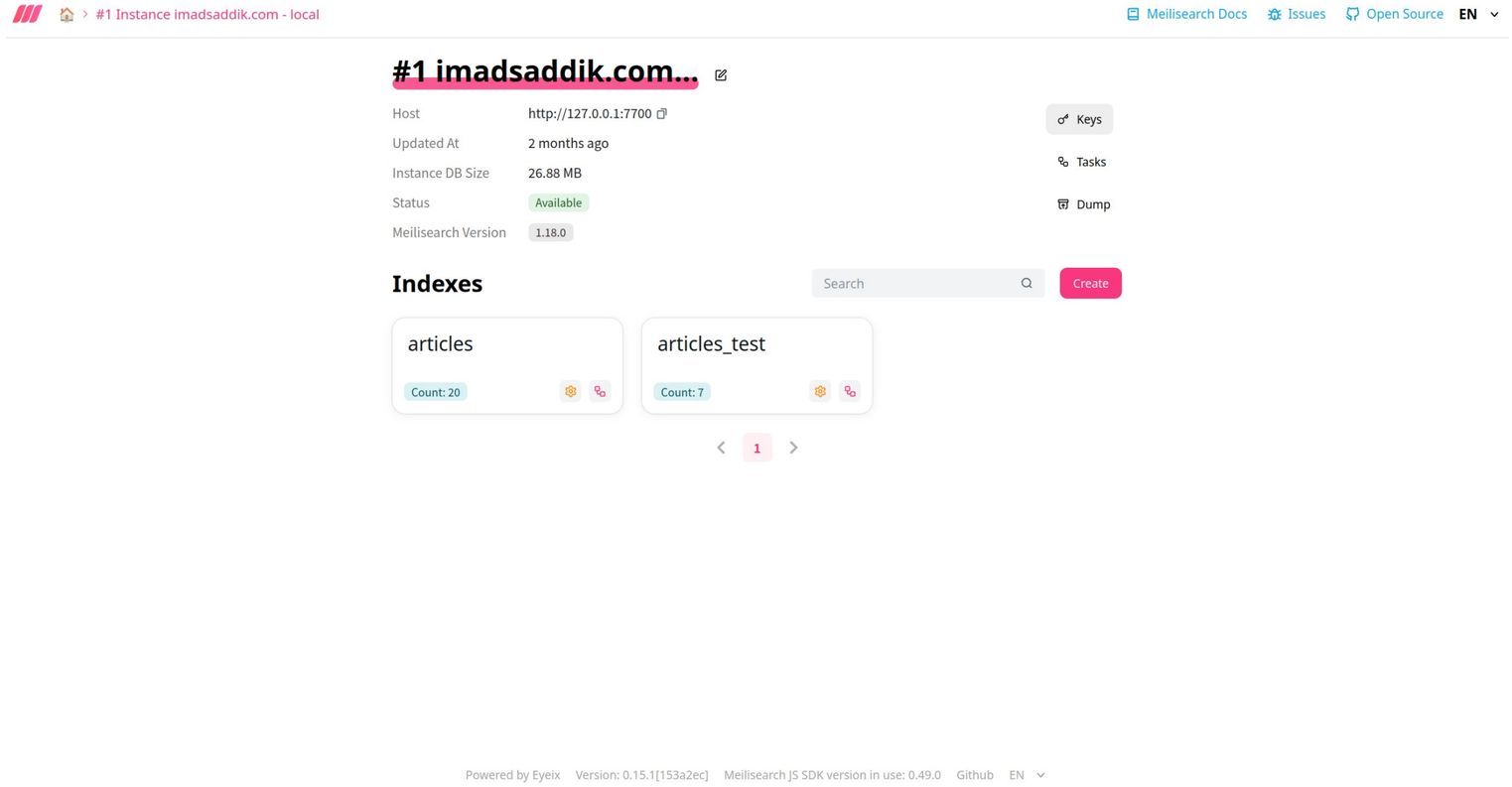
Application level

Operating System level



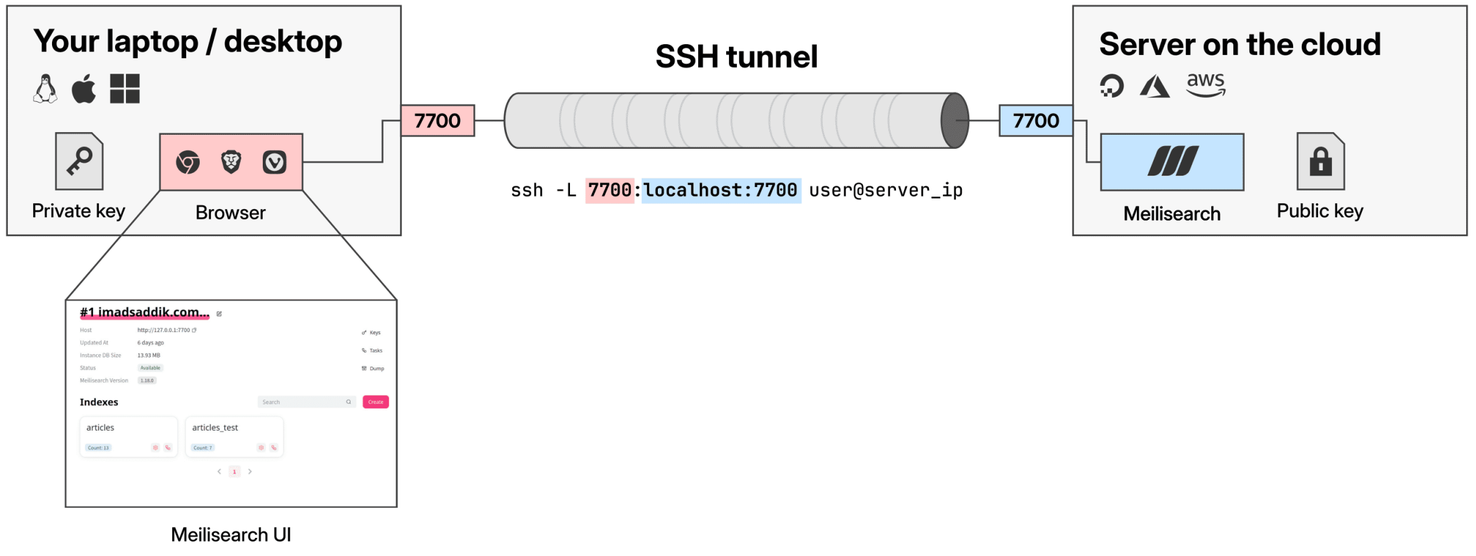
Secure database management

- **The tool**: meilisearch-ui (Open-source web dashboard).
- **Features**: Search, add, and edit documents visually.



Secure database management

- **The tool**: meilisearch-ui (Open-source web dashboard).
- **Features**: Search, add, and edit documents visually.
- **The connection**: Secure access via SSH Tunneling (no open ports).



Module 4: Global delivery and security

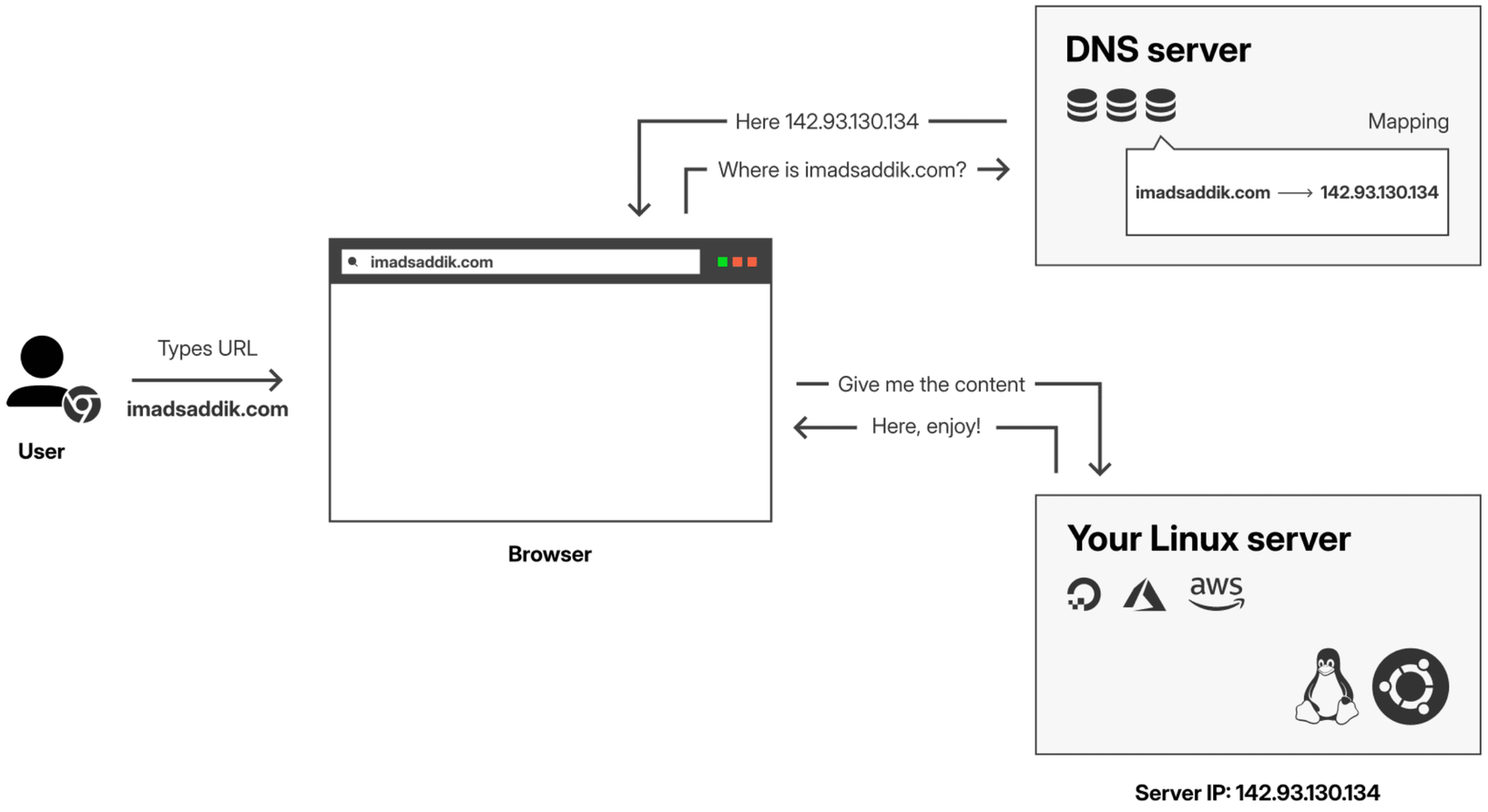
Domains and SSL

Domains & SSL

- Purchase a **custom domain**.
- Configure **DNS Records** (A and CNAME).
- Install free **SSL certificates** via Let's Encrypt.
- Automate 90-day **renewals** using Cron.



How DNS works

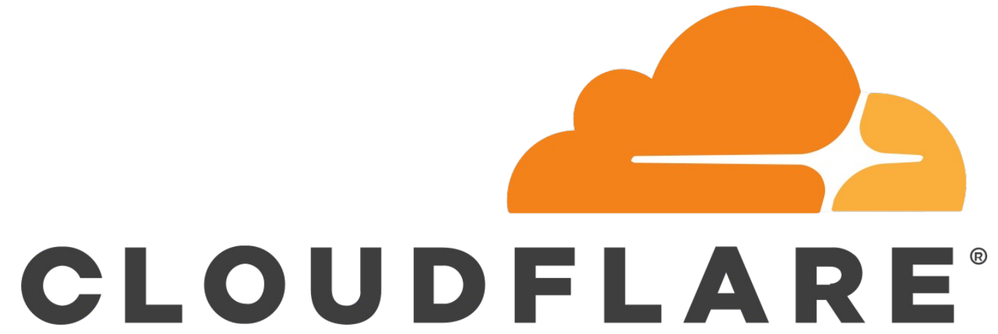


Module 4: Global delivery and security

The CDN layer and caching

Global speed & Caching

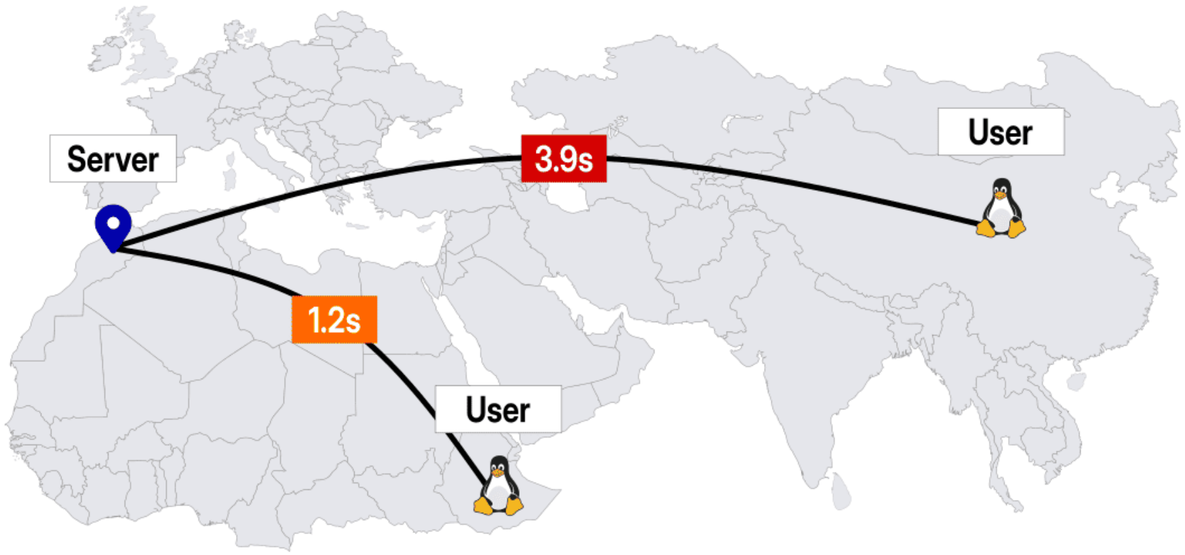
- Route traffic through a **Content Delivery Network (CDN)**.
- Configure strict **end-to-end SSL encryption**.
- Unmask visitor IP addresses in Nginx.
- Implement a **caching strategy** for SPAs.



Beating the speed of light

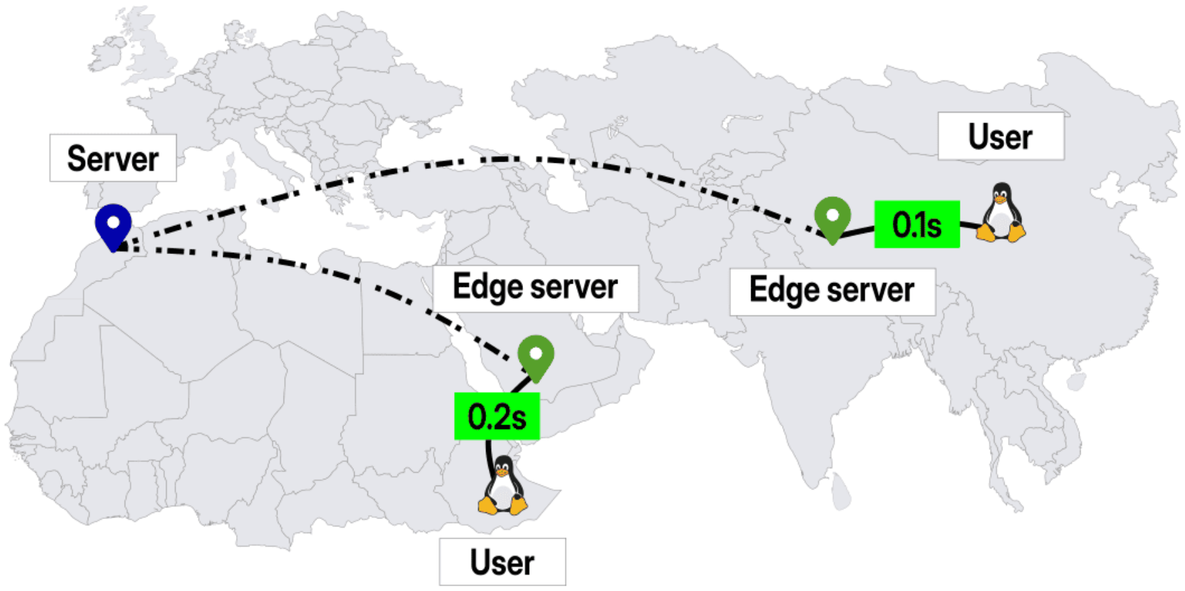
Before CDN

Users connect directly to your origin server. Physical distance causes slow load times for international visitors.



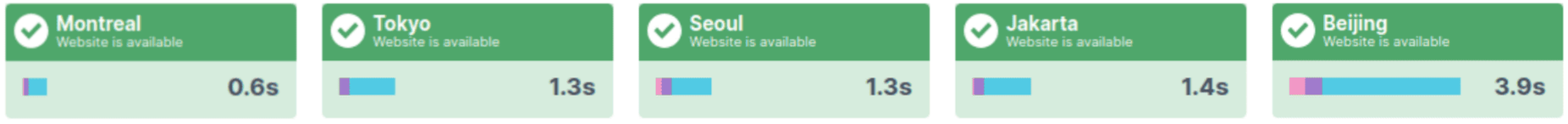
After CDN

A global network of edge servers caches your files. Users connect to a server right next to them, resulting in lightning-fast load times.

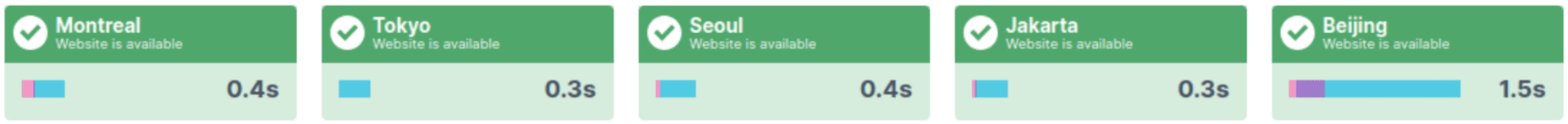


Beating the speed of light

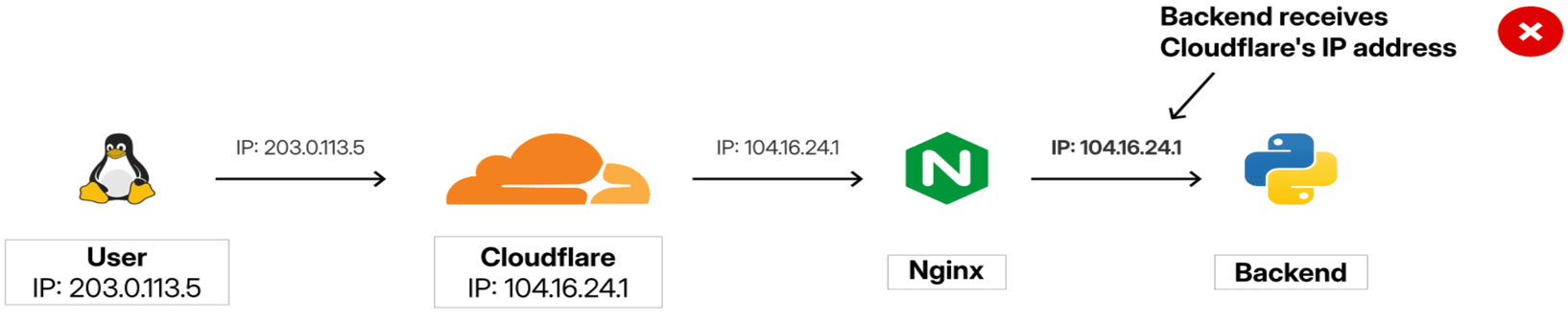
Before using Cloudflare



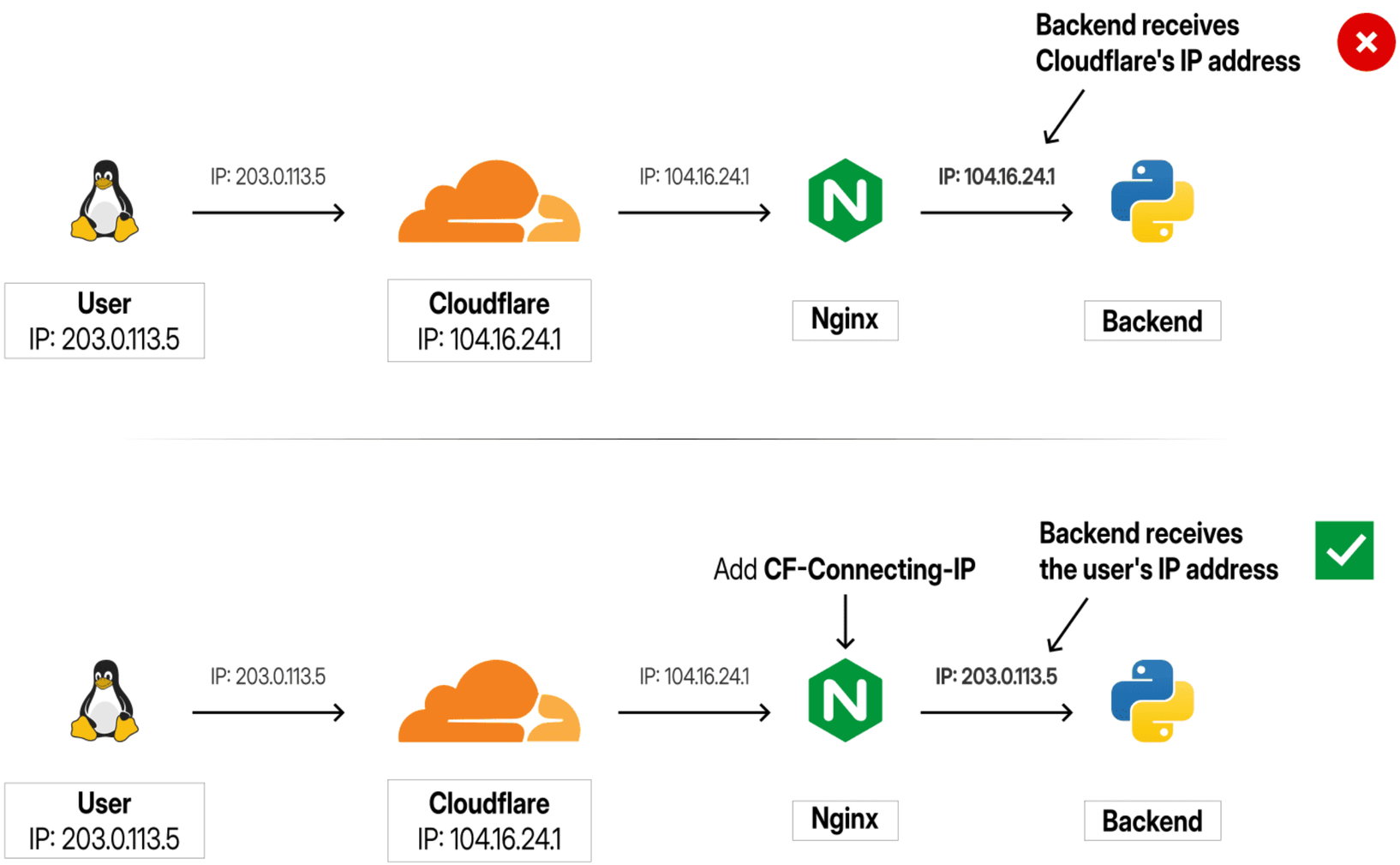
After using Cloudflare



The real IP problem



The real IP problem



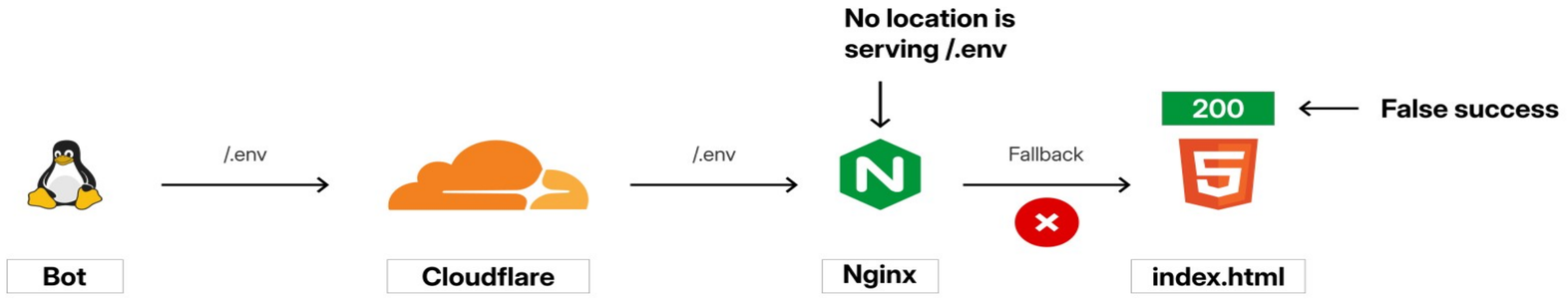
Module 4: Global delivery and security

Vulnerability scanning

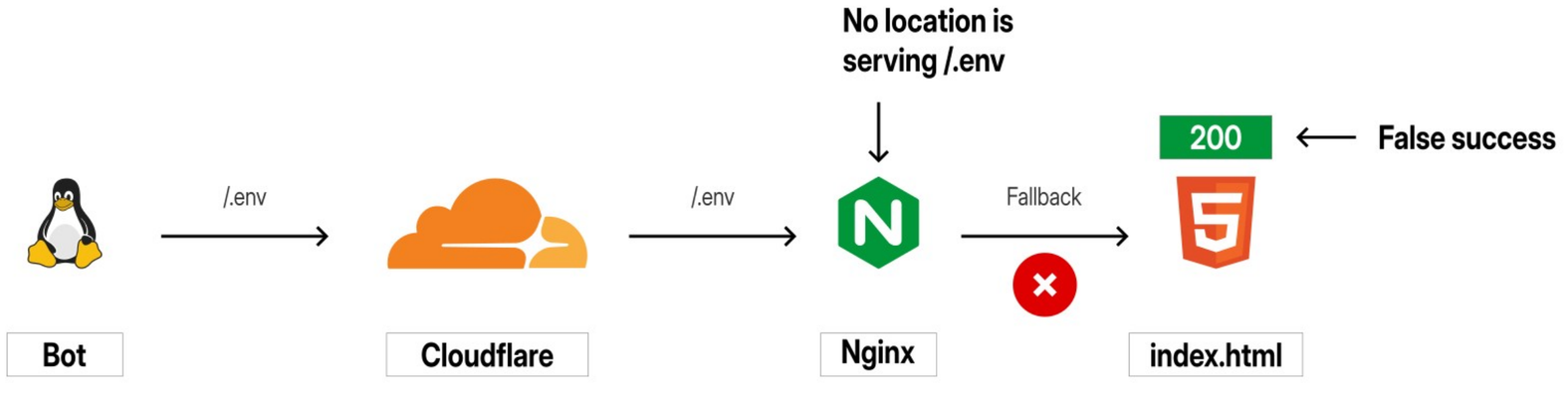
Testing your defenses

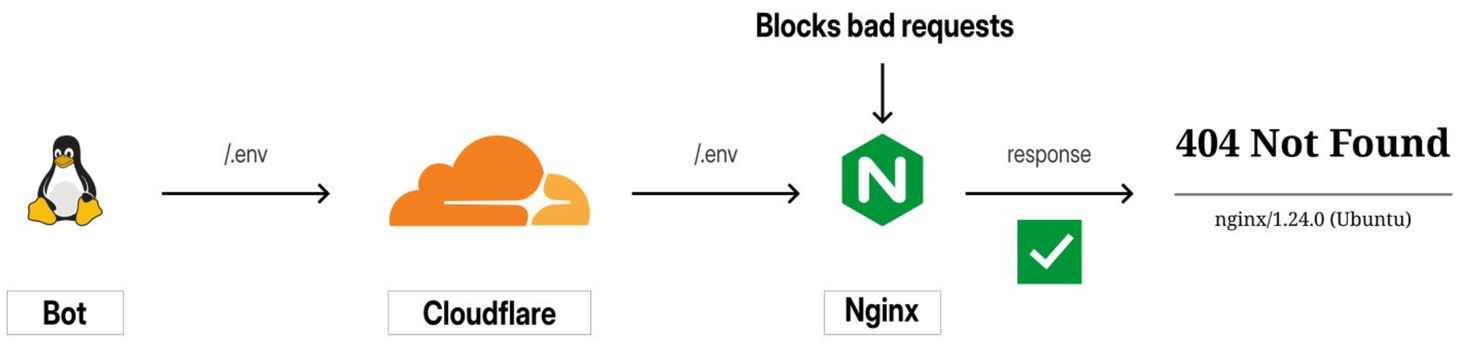
- Understand and fix the **Soft 404** vulnerability.
- Configure Nginx to instantly **block malicious bots**.
- Use nmap to **scan your firewall** from the outside.

Fixing the SPA trap (Soft 404s)



Fixing the SPA trap (Soft 404s)





Module 5: The automation pipeline

Local hygiene

Securing your local workflow

- Protect the **main** branch using **GitHub Rulesets**.
- Automate local checks with **pre-commit**.
- Lint and format Python using **Ruff**.
- Lint and format Vue.js using **ESLint & Prettier**.

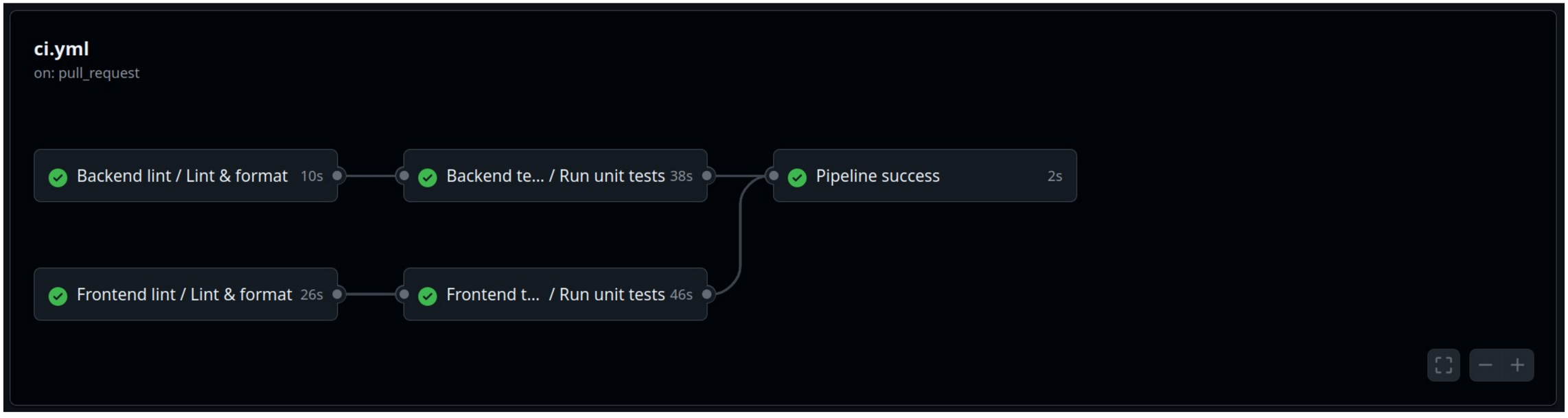
Module 5: The automation pipeline

Continuous integration

The cloud gatekeeper

- What is Continuous Integration (CI)?
- Build modular, reusable workflows.
- Automate linting, formatting, and unit tests.
- Block bad code using GitHub Status Checks.

The pipeline



Module 5: The automation pipeline

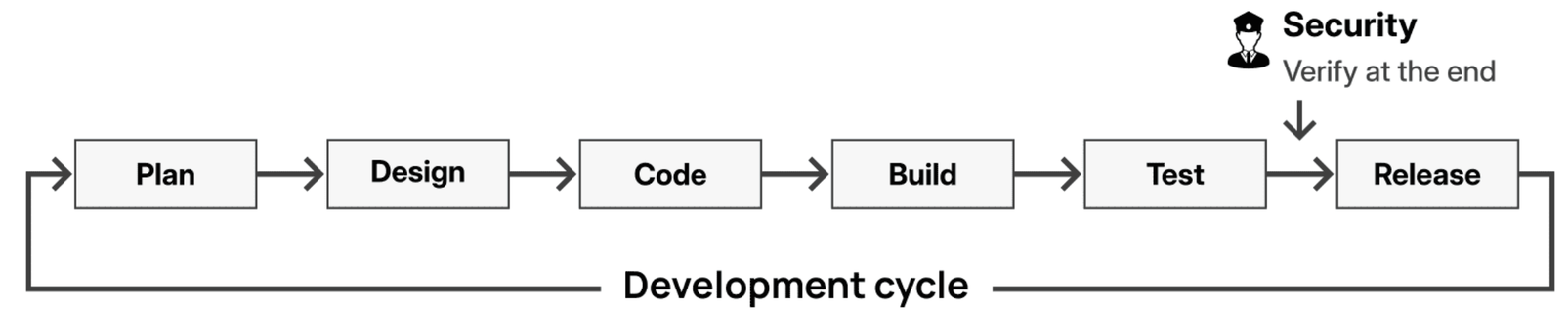
Automated security scanning

The security guardrails

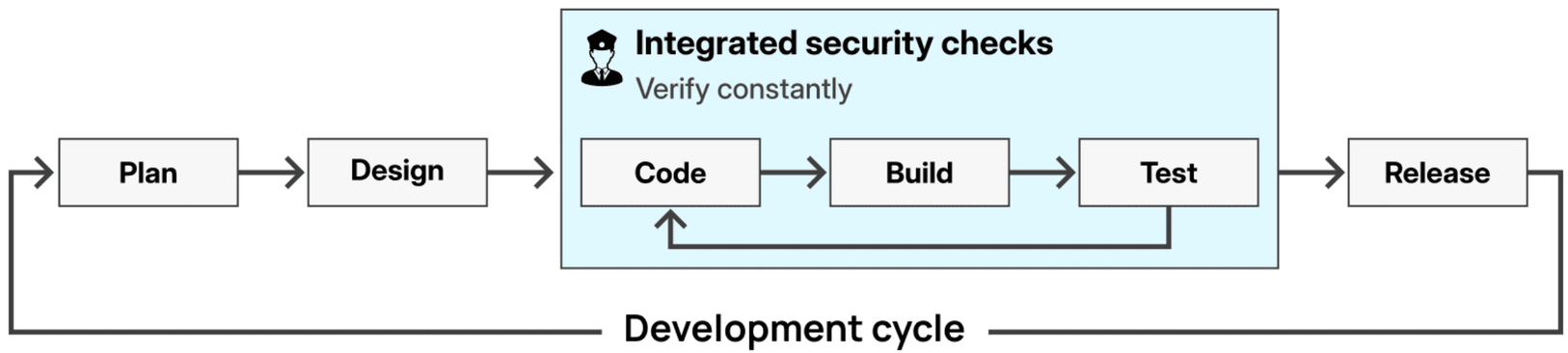
- Understand the **"Shift Left"** security model.
- Run Software Composition Analysis (**SCA**).
- Run Static Application Security Testing (**SAST**).
- Implement a **"Fail Fast"** pipeline strategy.

Shifting security left

Traditional security model



Shifting left security model



Module 5: The automation pipeline

Advanced testing & DAST

Testing a living system

- Write **integration tests** with a live Meilisearch instance.
- Simulate real users with **Playwright E2E tests**.
- Run Dynamic Application Security Testing (**DAST**) with **OWASP ZAP**.

Module 5: The automation pipeline

Continuous delivery

Automating production deployments

- Compile and upload the static Vue frontend.
- Trigger automated database and search engine **backups**.
- Require **human review** using GitHub Environments.
- Execute an **atomic** virtual environment **rollback strategy**.

Module 6: Optimization & maintenance

Observability

Moving beyond blind deployments

- Parse native Nginx traffic data with **GoAccess**.
- Protect server metrics using **HTTP Basic Authentication**.
- Handle Linux log rotation loops using **Cron**.
- Visual CPU, memory, and process trees using **Btop**.
- Isolate traffic outliers and apply network bans.

Module 6: Optimization & maintenance

Reliability

Scaling under pressure

- Simulate concurrent user traffic locally using **Locust**.
- Boost execution capacity **5x** with **FastHttpUser**.
- Analyze on-disk file storage sizes using **NCDU**.
- Enforce memory constraints on **systemd-journald**.
- Build a monthly automated cache and package cleanup system.

Module 6: Optimization & maintenance

Continuous security

Dynamic system defenses

- Move beyond static analysis and isolated code checks.
- Build a complete, synchronized stack inside a cloud runner.
- Run deep dynamic vulnerability testing with **OWASP ZAP**.
- Configure automated overnight schedules using **Cron**.
- Stabilize false alarms using a centralized rules whitelist.

Conclusion

From bare metal to autonomous infrastructure

Foundation (Ch1 & 2)

- Hardened SSH access
- UFW Firewall setup
- Nginx socket proxying
- Gunicorn process tree

Automation & security (Ch5 & 6)

- Pre-commit hooks
- CI/CD testing pipelines
- Atomic rollbacks
- Automated DAST scans and security audits

Performance & data (Ch3 & 4)

- Isolated Meilisearch search engine
- GoAccess real-time analytics
- Cloudflare CDN & SSL

Ready for production

- **Observable:** Real-time metrics and clean server logs.
- **Automated:** Zero-downtime deployments with instant rollbacks.
- **Secured:** Continuous vulnerability scanning and shift-left testing.